

Welcome to Slidev

Presentation slides for developers

Press Space for next page →



What is Slidev?

Slidev is a slides maker and presenter designed for developers, consist of the following features

-  **Text-based** - focus on the content with Markdown, and then style them later
-  **Themable** - theme can be shared and used with npm packages
-  **Developer Friendly** - code highlighting, live coding with autocompletion
-  **Interactive** - embedding Vue components to enhance your expressions
-  **Recording** - built-in recording and camera view
-  **Portable** - export into PDF, PNGs, or even a hostable SPA
-  **Hackable** - anything possible on a webpage

Read more about [Why Slidev?](#)

agenda

- 1. [Welcome to Wiremock](#)
- 2. [What is Slidev?](#)
- 3. [agenda](#)
- 4. [wiremock](#)
- 5. [wiremock-stubbing](#)
- 6. [wiremock-request-matching](#)
- 7. [wiremock-response-templating](#)
- 8. [wiremock-record-playback](#)

22

- 9. [wiremock-simulating-faults](#)
- 10. [wiremock-extending-wiremock](#)
- 11. [wiremock-apis](#)
- 12. [Extending-GlobalSettingsListener](#)
- 13. [Extending-MappingsLoaderExtension](#)
- 14. [Extending-RequestFilter](#)
- 15. [Extending-RequestMatcherExtension](#)
- 16. [Extending-ResponseDefinitionTransformer](#)

- 17. [Extending-ResponseTransformer](#)
- 18. [Extending-ServeEventListener](#)
- 19. [Extending-StubLifecycleListener](#)
- 20. [Extending-StubRequestFilter](#)
- 21. [Extending-TemplateHelperProviderExtension](#)
- 22. [Extending-TemplateModelDataProviderExtension](#)

wiremock

WireMock supports several approaches for creating mock APIs - in code, via its REST API, as JSON files and by recording HTTP traffic proxied to another destination. WireMock has a rich matching system, allowing any part of an incoming request to be matched against complex and precise criteria. Responses of any complexity can be dynamically generated via the Handlebars based templating system. Finally, WireMock is easy to integrate into any workflow due to its numerous extension points and comprehensive APIs.

Key features

- HTTP response stubbing, matchable on URL, header and body content patterns
- Request verification
- Runs in unit tests, as a standalone process or as a WAR app
- Record/playback of stubs
- Configurable response delays and Fault injection
- Per-request conditional proxying
- Browser proxying for request inspection and replacement
- Stateful behaviour simulation

wiremock-stubbing

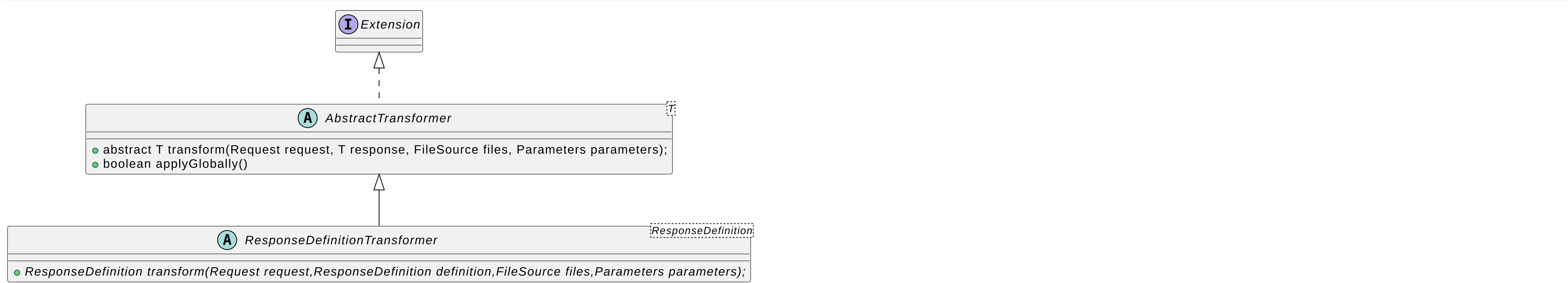
wiremock-request-matching

Stub matching and verification queries can use the following request attributes:

- URL
- HTTP Method
- Query parameters
- Form parameters
- Headers
- Basic authentication (a special case of header matching)
- Cookies
- Request body
- Multipart/form-data

wiremock-response-templating

```
WireMockServer wm = new WireMockServer(options().globalTemplating(true));
```



The request

- `request.pathSegments.[<n>]` - URL path segment (zero indexed)
- `request.query.<key>` - First value of a query parameter
- `request.headers.[<key>]` - Header with awkward characters
- `request.cookies.<key>` - First value of a request cookie
- `request.body` - Request body text

- Handlebars helpers
- Date and time helpers
- Random value helper
- Formatting numbers
- XPath helpers

wiremock-record-playback

wiremock-simulating-faults

Global fixed stub delays

```
{
  "fixedDelay": 500
}
```

Available distributions

- Lognormal delay

```
{
  "delayDistribution": {
    "type": "lognormal",
    "median": 80,
    "sigma": 0.4
  }
}
```

- Uniform delay

```
{
  "delayDistribution": {
    "type": "uniform",
    "lower": 15,
    "upper": 25
  }
}
```

Chunked Dribble Delay

```
{
  "chunkedDribbleDelay": {
    "numberOfChunks": 5,
    "totalDuration": 1000
  }
}
```

wiremock-extending-wiremock

Each extension point is defined by an interface that extends from Extension and extension implementations are loaded at startup time.

At present, the following extension interfaces are available:

RequestFilterV2 / AdminRequestFilterV2 / StubRequestFilterV2 : Intercept requests, modifying them or taking alternative actions based on their content.

ResponseDefinitionTransformerV2 : Modify the response definition used to generate a response. See Transforming responses.

ResponseTransformerV2 : Modify the response served to the client. See Transforming responses.

ServeEventListener : Listen for events at various points in the request processing lifecycle. See Listening for Serve Events.

AdminApiExtension : Add admin API functions. See Admin API Extensions.

RequestMatcherExtension : Implement custom request matching logic. See Custom matching.

GlobalSettingsListener : Listen for changes to the settings object. See Listening for Settings Changes.

StubLifecycleListener : Listen for changes to the stub mappings. See Listening for Stub Changes.

TemplateHelperProviderExtension : Provide custom Handlebars helpers to the template engine. See Adding Template Helpers.

TemplateModelDataProviderExtension : Provide additional data to the model passed to response templates. See Adding Template Model Data.

MappingsLoaderExtension : Provide additional source to load the stub mappings. See Adding Mappings Loader.

Registering Extensions

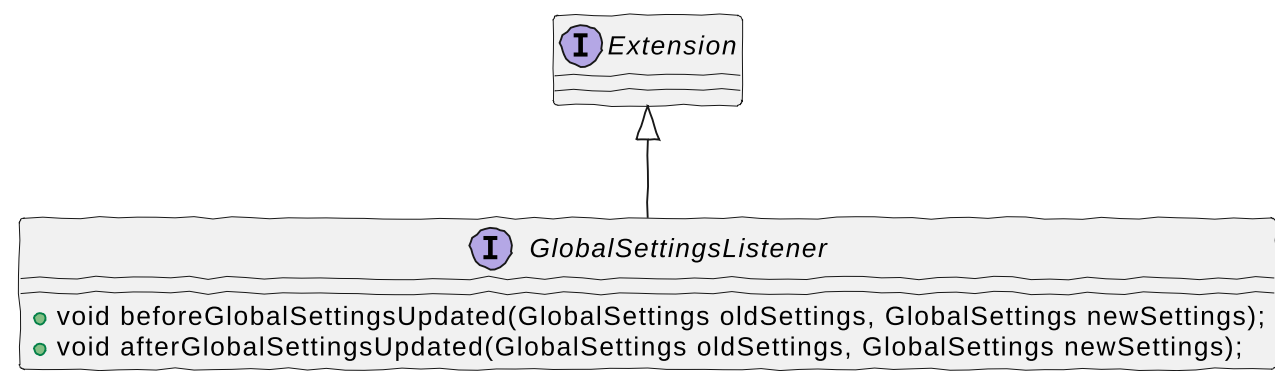
```
new WireMockServer(wireMockConfig().extensions(BodyContentTransformer.class, HeaderMangler.class));
```

WireMock

/_admin

- Stub Mappings
- Requests
- Recordings
- Scenarios
- Files
- System

Extending-GlobalSettingsListener



Extending-MappingsLoaderExtension

I

Extension

String getName();

void start();

void stop();

I

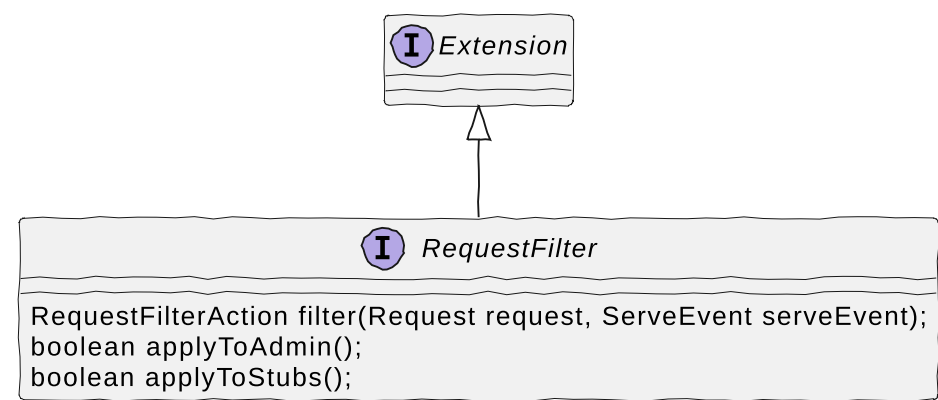
MappingsLoader

void loadMappingsInto(StubMappings stubMappings);

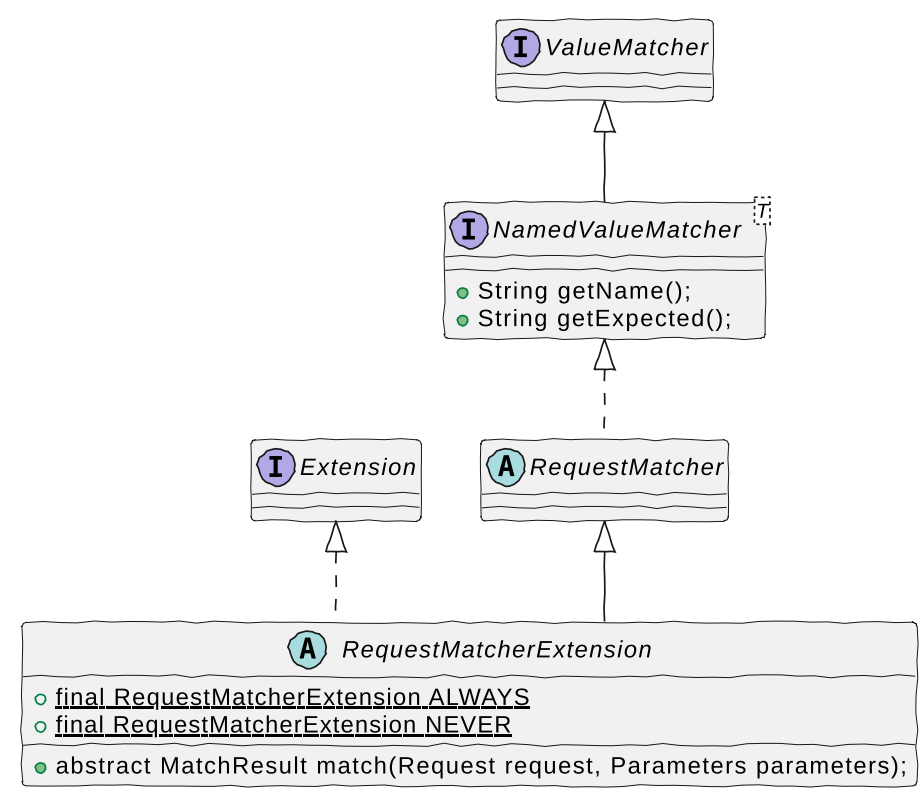
I

MappingsLoaderExtension

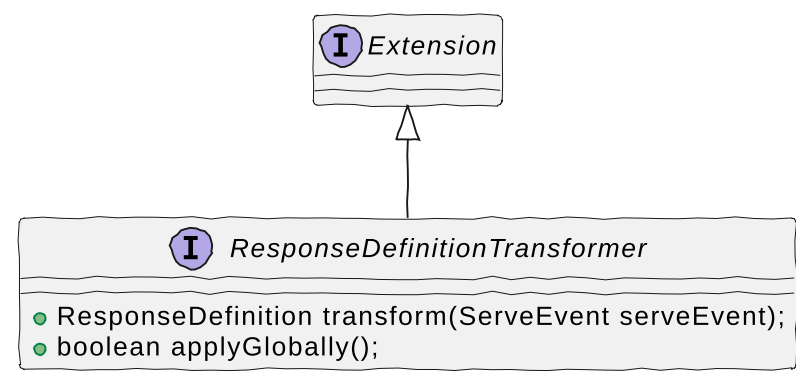
Extending-RequestFilter



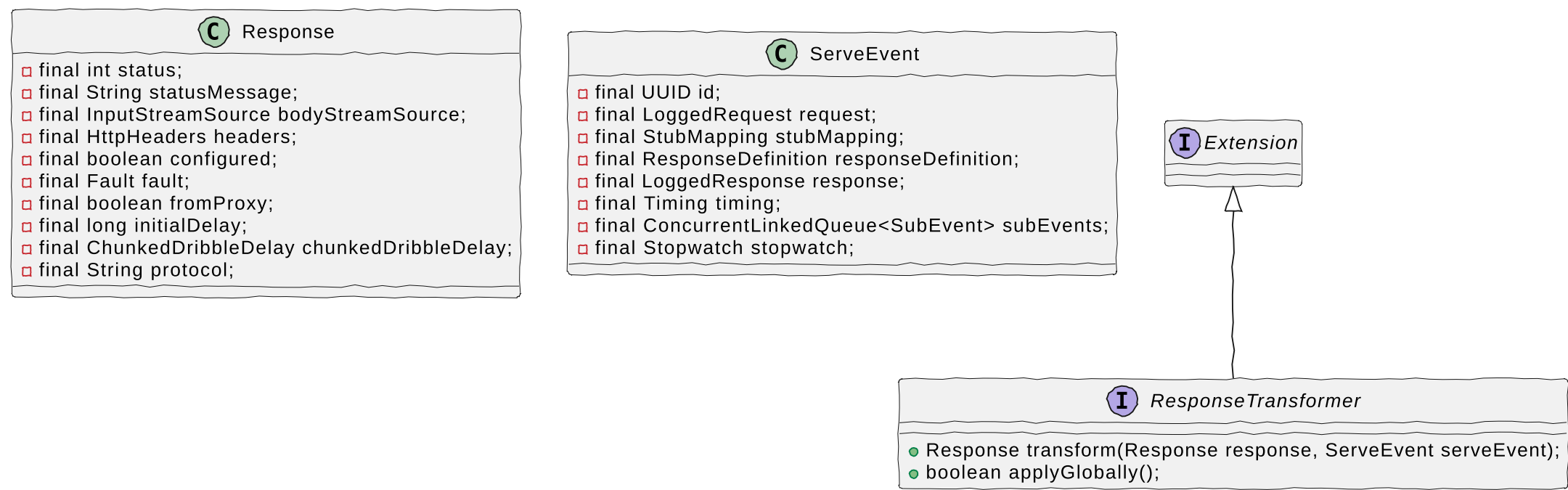
Extending-RequestMatcherExtension



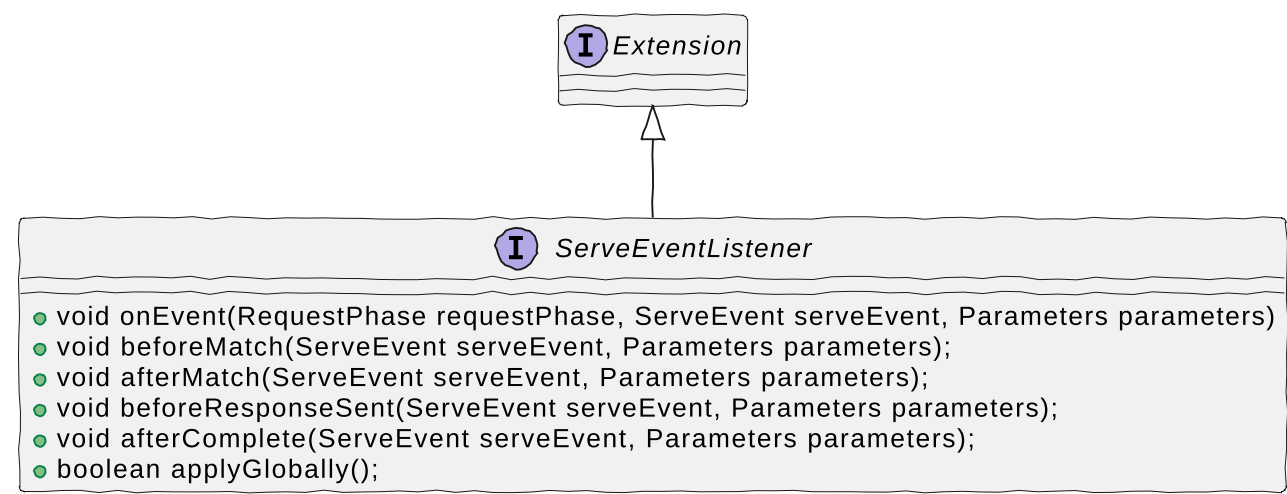
Extending-ResponseDefinitionTransformer



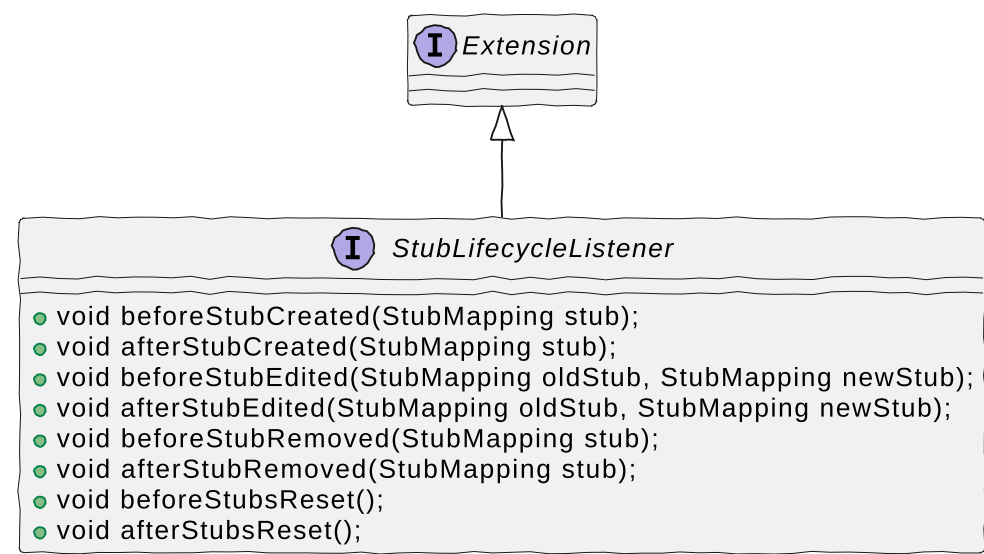
Extending-ResponseTransformer



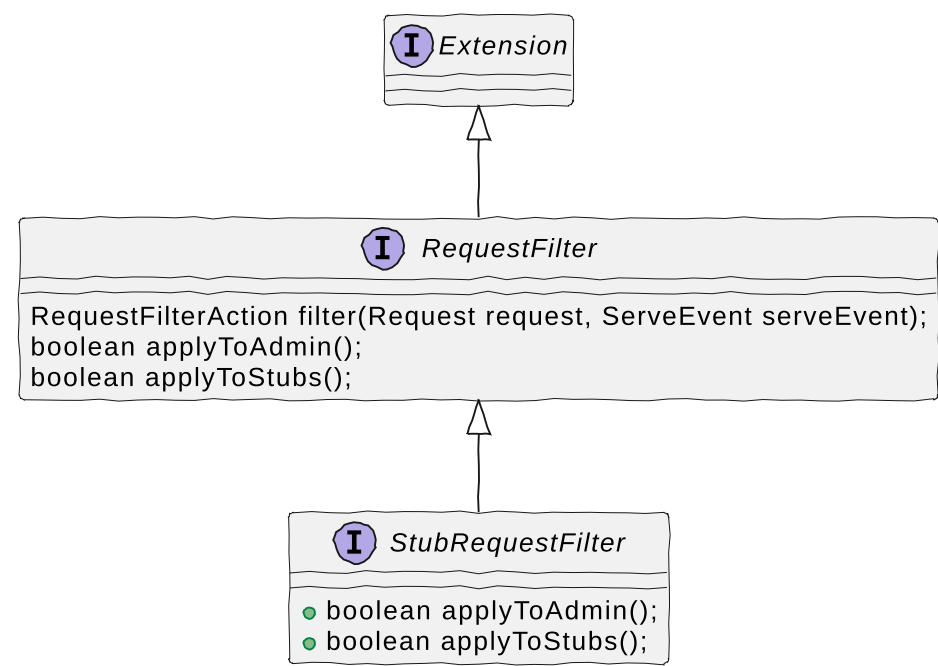
Extending-ServeEventListener



Extending-StubLifecycleListener



Extending-StubRequestFilter



Extending-TemplateHelperProviderExtension

